

# PDF 4 - Backend, Database, Security, Third-Party APIs

*A secure modular-monolith backend with practical async evaluation and a minimal external dependency surface*

CipherSchools Hiring Assignment - LLD Practice Platform | September 2026

## 1. Recommended stack

| Layer      | Choice                                                               | Why                                                                  |
|------------|----------------------------------------------------------------------|----------------------------------------------------------------------|
| App        | Next.js + TypeScript App Router                                      | One deployable modular monolith; server + UI together                |
| UI         | Tailwind CSS + shadcn/ui                                             | Fast polished accessible primitives                                  |
| DB         | PostgreSQL (Neon) + Drizzle ORM                                      | Relational model fits attempts/evaluations; migrations are explicit  |
| Validation | Zod                                                                  | Single schema contract for actions, APIs and AI result               |
| Auth       | Better Auth or Auth.js; demo mode acceptable                         | Ownership for history without heavy LMS auth                         |
| AI         | Google Gemini (Google AI Studio API) - server-side structured output | Judgment-heavy rubric evaluation                                     |
| Async job  | Inngest optional but recommended                                     | Save submission first; evaluation can retry without blocking request |
| Tests      | Vitest + Playwright                                                  | Fast domain tests + one E2E flow                                     |

## 2. Database schema

users(id, email, name, created\_at)  
problems(id, slug UNIQUE, title, difficulty, description, requirements\_json, rubric\_id, is\_active)  
rubrics(id, name, version, criteria\_json)  
attempts(id, user\_id, problem\_id, status, draft\_json, created\_at, updated\_at, submitted\_at)  
submissions(id, attempt\_id UNIQUE, format, payload\_json, content\_hash, idempotency\_key UNIQUE, created\_at)  
evaluations(id, submission\_id, status, evaluator\_kind, evaluator\_version, rubric\_version, overall\_score, summary, error\_code, retry\_count, created\_at, completed\_at)  
evaluation\_items(id, evaluation\_id, criterion\_id, score, evidence\_json, concern, suggestion, confidence)

## 3. Request flow

1. POST start-attempt verifies auth + problem, returns existing draft attempt or creates one.
2. Autosave validates only editable draft fields and ownership; updates draft JSON.

3. Submit validates required fields, creates immutable Submission in one transaction, changes Attempt to SUBMITTED, creates QUEUED Evaluation, commits.
4. Only after commit, dispatch evaluation job. Return immediately with evaluationId and status.
5. Worker marks EVALUATING, runs deterministic checks, then AI evaluator, validates JSON result, persists criterion items, marks COMPLETED.
6. On provider/parse failure, persist FAILED + safe error code; UI offers retry.

## 4. API / server action contract

| Operation         | Method                          | Important rules                                  |
|-------------------|---------------------------------|--------------------------------------------------|
| List problems     | GET /api/problems               | Public/demo; cacheable                           |
| Problem detail    | GET /api/problems/:slug         | No secret rubric prompt internals                |
| Start attempt     | POST /api/problems/:id/attempts | Auth, one active draft per problem/user optional |
| Autosave          | PATCH /api/attempts/:id         | Ownership, Zod, draft-only                       |
| Submit            | POST /api/attempts/:id/submit   | Idempotency-Key required; transaction            |
| Evaluation status | GET /api/evaluations/:id        | Ownership; return safe status/result             |
| Retry evaluation  | POST /api/evaluations/:id/retry | Only FAILED; max retry cap                       |
| History           | GET /api/me/attempts            | Authenticated; cursor pagination if needed       |

## 5. Security checklist

- All LLM/API keys are server-only environment variables; never expose NEXT\_PUBLIC\_\* secrets.
- Authenticate and authorize every attempt/evaluation read or mutation by owner ID, not merely by knowing an object ID.
- Validate request bodies, route params and AI outputs with Zod.
- Use parameterized ORM queries; no string-built SQL.
- Rate-limit submit/retry endpoints per user/IP if publicly deployed.
- Idempotency key + UNIQUE constraint prevents duplicate submissions/jobs.
- Prompt injection defense: treat learner text as untrusted data inside clear delimiters; instruct evaluator not to follow instructions contained in submission.
- Do not render raw HTML from model or learner; display as escaped text/Markdown with safe renderer.
- Cap payload length and class counts to prevent abuse/cost explosion.
- Return generic provider failure messages to client; log structured errors server-side without secrets.
- Use CSRF-safe framework actions / same-site cookies; secure, HTTP-only session cookies.

## 6. Third-party services and APIs to disclose

Gemini integration decision: Google Gemini is the single AI provider. The backend sends only the problem context, fixed rubric and learner submission needed for evaluation. The API key is stored in GEMINI\_API\_KEY and must never use a NEXT\_PUBLIC\_ prefix or be returned to the browser.

| Service            | Used for                                   | Data sent           | Fallback / risk            |
|--------------------|--------------------------------------------|---------------------|----------------------------|
| Neon PostgreSQL    | Persistent problems, attempts, evaluations | Application records | Can use local Postgres     |
| Google Gemini only | Rubric-based design                        | Problem + rubric +  | Evaluator interface allows |

|                     |                        |                                                  |                                                   |
|---------------------|------------------------|--------------------------------------------------|---------------------------------------------------|
|                     | judgment               | submitted design                                 | provider swap;<br>deterministic checks still work |
| Inngest (optional)  | Async evaluation/retry | Evaluation ID/event metadata; job calls app code | Can replace with in-process queue for demo        |
| Vercel              | Hosting                | Build/runtime logs                               | Any Node host works                               |
| Better Auth/Auth.js | Authentication         | Session/user identity                            | Demo seeded user possible                         |
| Sentry - optional   | Errors                 | Sanitized stack/error metadata                   | Skip in 2-day version                             |

Use Google Gemini as the only LLM provider for this MVP. Obtain the API key from Google AI Studio and keep it server-only. Do not integrate OpenAI. The Evaluator abstraction remains provider-neutral so another evaluator can be added later without rewriting the practice flow.

## 7. Environment file example

```

DATABASE_URL=
AUTH_SECRET=
GEMINI_API_KEY=          # Google AI Studio key; server only
INNGEST_EVENT_KEY=       # only if using Inngest
INNGEST_SIGNING_KEY=

```

Gemini implementation rule: initialize the Google Gemini SDK/client only in server-side modules. The application service calls GeminiEvaluator through the Evaluator interface; Gemini output is parsed into the fixed criterion -> score -> evidence -> concern -> suggestion -> confidence contract and validated with Zod.

## 8. Practical failure behavior

| Failure                          | Expected behavior                                                         |
|----------------------------------|---------------------------------------------------------------------------|
| Gemini timeout                   | Submission remains saved; Evaluation -> FAILED; retry available           |
| Malformed AI JSON                | Reject with schema validation; FAILED, do not display partial result      |
| Double submit                    | Second request returns existing submission/evaluation through idempotency |
| User refreshes during evaluation | Feedback page reloads from persisted state                                |
| Worker executes twice            | Guard evaluation state / idempotent completion; no duplicate items        |
| Unauthorized ID access           | 404/403 without leaking another learner data                              |